Triody AI: Technical Documentation

Topic E: SME Finance Copilot | 2026 Shenzhen International FinTech Competition, International Track

Team Triody AI, Edinburgh Napier University | October 2026

1. What Triody does

Triody AI is a finance copilot for a small business owner. You give it the shop's transactions, the bills you know are coming and a short business profile. It forecasts cash for the next 60 days, warns you before the balance drops under your safety buffer, explains why, and answers questions about sales, costs and profit for any period. When cash runs short, it ranks financing options from public sources, tests the best one against the forecast and drafts the application. You approve every action with a one-time code. Triody never moves money.

The code is Python 3.11 with the standard library only. The repository holds four synthetic shops, all fictional: a café in Edinburgh (shop A), a bike shop (B), a florist (C) and a tea shop in Shenzhen priced in yuan (D).

2. System architecture

Every request follows one path. The interface sends the message to `webapi.dispatch`, which checks method, size and JSON shape. The Copilot asks the Brain for a plan, then runs each tool call through the policy gate. Tools read from the engines and the data. Every step goes into the audit log. Approval codes travel on a separate channel to the owner device and never enter the chat.

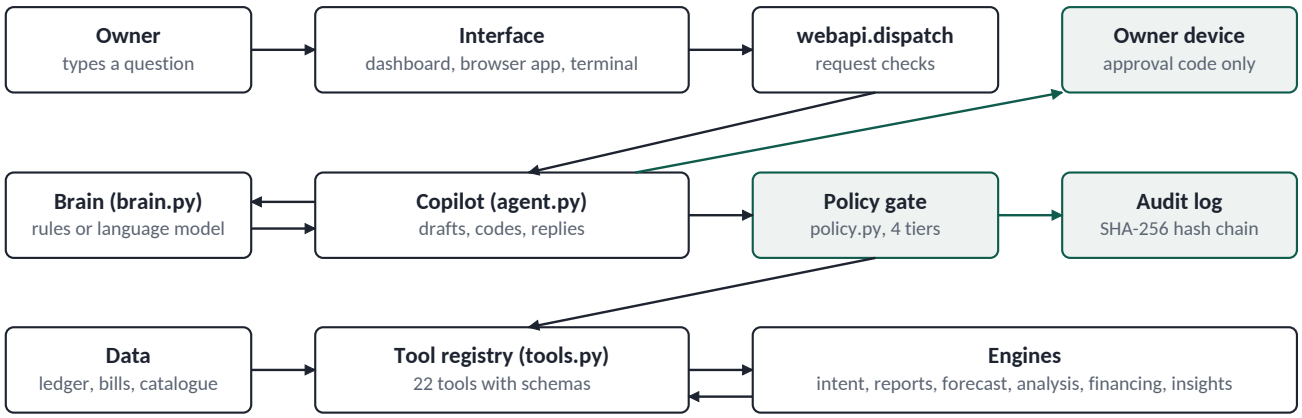


Figure 1. Request path. Green boxes are the security controls.

Module	Job
`agent.py`	The Copilot. Runs tools through the gate, keeps drafts and codes, writes replies, runs the daily check.
`brain.py`	Picks the tools. RuleBrain works offline. LLMBrain uses Anthropic's API or any OpenAI-compatible server, such as Ollama.
`intent.py`	Reads the message: fixes typos, finds the period and the metric, picks one intent.
`policy.py`	Permission tiers, one-time codes, amount and daily limits.
`tools.py`	Registry of 22 tools. Each tool has one tier and an argument schema.
`forecast.py`	Sales model, recurring bill detection, Monte Carlo cash forecast, backtest.
`financing.py`	Hard filters, scoring and the affordability cap for financing products.
`analysis.py`, `reports.py`, `insights.py`	Period figures, monthly analysis, shortfall reasons, bills, health check, what-if tests.
`audit.py`	Append-only, hash-chained audit log with redaction.
`access.py`, `server.py`, `webapi.py`	Local server, device pairing, sessions, shared request handling.

Module	Job
`region.py`	Country settings. A shop picks the UK or China catalogue in its profile.

You run Triody in one of three ways:

- **Browser app.** The `webapp/` folder runs the same Python package in the page through Pyodide 0.27.7. A service worker caches it, so it works offline after the first visit and installs on a phone.
- **Local dashboard.** `run_windows.bat` or `run_mac.command` runs the tests, then serves the dashboard at 127.0.0.1:8000.
- **Phone over Wi-Fi.** `python -m triody serve --lan` lets a phone on the same network sign in once with a 6-digit pairing code shown on the computer.

Each shop folder holds four files: `transactions.csv` (date, category, counterparty, amount, memo), `commitments.json`, `business_profile.json` and `meta.json`.

3. Core algorithms

3.1 Reading the question

- Typos are fixed against a finance vocabulary with optimal string alignment distance, so "slaes" reads as "sales".
- Periods (today, this month, a quarter, a named month, explicit dates) count from the last day in the books, not the computer clock. The same ledger gives the same answer on any day.
- An unclear message gets a question back with three suggested questions to tap. It never triggers an action.

3.2 Cash-flow forecast

- **Sales model.** A weekday pattern from the last 28 days. A weekday under 5% of the average counts as closed and stays at zero. Any day above 1.5 times the median is capped, so one wedding order does not lift the level. The level uses exponential smoothing with a 7-day half-life. A linear trend keeps 60% of its strength each week.
- **Recurring bills.** A payee seen at least 3 times on a weekly, two-weekly or monthly cycle gets projected to its next dates. Weekly supplier bills follow the previous 7 days of sales. Irregular spend gets spread over each day at its long-run rate. Bills you list, such as VAT, go in on their dates.
- **Monte Carlo.** 2,000 simulated futures over 60 days. Each draws its own trend, drift, level shock, daily noise and bill amounts. Daily noise comes from up to 84 days of history, one-offs included.
- **Self-calibration.** Triody refits the model on each past day of the last 90 days, forecasts 14 days and measures its own error. The spread of those errors sets the range width, with a floor of 0.14.
- **Alerts.** Critical when the median path goes below zero, or when 50% or more of futures do. Warning when the chance of dropping under the buffer reaches 25% on any day.

3.3 Financing matcher

The UK catalogue holds 10 products and the China catalogue holds 4. Each product lists its source link, the date checked and every assumption. The UK list includes HMRC Time to Pay, a Lloyds overdraft, iwoca, Funding Circle and the Start Up Loan. The China list includes WeBank Weiyedai.

- **Step 1, hard filters.** Amount, term, months trading, revenue, legal form, existing bank, credit band, security, card sales, invoices, purpose. A rejected product keeps its reason, for example "The minimum amount is £10,000, you need £5,000."
- **Step 2, score from 0 to 100.** Cost 35%, affordability 25%, speed 20%, purpose fit 10%, risk 10%. Speed loses 15 points for each day past your deadline. Affordability scores 100 up to 30% of monthly free cash flow and 0 at 100%.
- **Step 3, affordability cap.** Triody recommends a product only if the monthly repayment stays within 50% of average monthly free cash flow (net cash over the last 180 days). It first tries longer terms inside the product's limits. If nothing fits, it recommends nothing and says why.
- **Stock rule.** For stock bought on terms of 12 months or less, the stock's own sales repay most of the loan. The cap counts the cost plus 25% of the amount in case some stock does not sell.
- **Sizing.** The need equals the expected gap to the buffer plus 25%, rounded up to the next £500. The deadline is the first risk day minus 3 days.

3.4 Plan test

Before drafting anything, Triody adds the repayment schedule to your commitments and reruns the forecast: 60 days with 2,000 futures, then the whole plan up to 366 days with 600 futures. In the café demo, HMRC Time to Pay on £5,000 over 10 months cuts the 60-day chance of going below zero from 66% to 13%. Over the whole plan, the chance only falls from 73% to 59%. Triody tells the owner the plan buys time and does not close the gap on its own.

3.5 Number grounding

When a language model writes the reply, `numbers_grounded` checks every number of 100 or more against the tool results. Years from 1900 to 2100 get skipped. A figure no tool returned gets its wording replaced, so the model cannot invent a balance.

4. Security design

Code enforces every rule below. The system prompt asks a model to treat data as data, but Triody does not rely on it.

4.1 Permission tiers

Tier	Tools	Rule in code
READ	17, for example forecast_cash, match_financing, check_plan, period_report	Always allowed. Arguments checked against the schema.
PROPOSE	2: draft_financing_application, draft_supplier_delay	Always allowed. Eligibility and supplier name checked again. Nothing leaves the sandbox.
EXECUTE	2: submit_financing_application, schedule_supplier_delay	Only with origin "owner" and a redeemed one-time code. Refused for the agent.
PROHIBITED	1: transfer_funds	Refused for every origin, even with a code. The function body raises an error if reached.

A language model sees only READ and PROPOSE tools. Two tests check the EXECUTE and PROHIBITED tools stay off the list, one of them on the payload sent to the model.

4.2 Approval flow

- A draft gets a random 48-bit code (`secrets.token_hex(6)`). The code binds to the SHA-256 hash of the exact tool and arguments, expires after 10 minutes and works once.
- The code goes to the owner device panel only. The chat, the model, the conversation memory and the audit log never hold it.
- Before running, Triody checks the limits: £50,000 per action and 3 executed actions per day.
- Five wrong codes lock every waiting draft. Cancelling revokes the code.
- The daily check runs once per session. It calls READ tools and at most one PROPOSE tool, then stops. It never redeems a code.

4.3 Model containment

- Memo text from the ledger gets hidden from the model, which blocks prompt injection through a supplier memo.
- Small local models sometimes write tool calls as text. Triody reads those for offered tools only, through the same gate.
- The model gets 6 planning steps at most. If it goes offline, the rule planner answers and the event goes into the log.
- The browser app never uses a model, since a key in a public page would be readable.

4.4 Audit log

- Each entry stores the SHA-256 hash of the previous entry plus its own body, starting from a block of zeros. `AuditLog.verify` finds the first broken link. The demo log passes on all 41 entries.
- E-mail addresses and digit strings of 8 or more get masked before logging. Codes get stored as "[code]".
- An optional tool, `tools/anchor_audit.py`, writes the chain head to the Sepolia test network, so a full rewrite of the file shows up.

4.5 Server and devices

- The server binds to 127.0.0.1, serves a fixed file list, needs a custom header on every POST, caps the body at 4,000 bytes and sends a strict Content-Security-Policy.
- The Host header must be localhost or a plain IP address, which blocks DNS rebinding.

- Pairing codes have 6 digits, work once and last 10 minutes. Five wrong tries lock a device out for 5 minutes. Ten wrong tries replace the code.
- Sessions use a random HttpOnly, SameSite=Strict cookie for 30 days. Triody stores only its SHA-256 hash, so a copied session file cannot get replayed.

5. Testing and results

The repository has 192 unit, integration and safety tests, and all 192 pass. You run them with `python -m unittest discover -s tests -t .` and rebuild every number below with `python -m triody evidence`.

Test	Triody	Baseline	Notes
Matcher, 30 labelled cases	86.7% strict, 6/6 correct refusals	76.7%, 4/6 (cheapest eligible)	Labels written before the matcher ran
Matcher, 14 held-out cases, UK catalogue	85.7% strict, 100% relaxed, 3/3 refusals	71.4%, 1/3	First run, no label changed
Matcher, 15 new held-out cases	93.3% strict, 2/2 refusals	86.7%, 1/2	Written 14 Oct 2026, frozen before the run
Forecast, day-14 balance error, shop A / B / C / D	£672 / £1,434 / £524 / ¥8,060	£1,441 / £2,721 / £793 / ¥25,793 (naive)	Triody wins on all four shops
Forecast, weekly sales error, shop A / B / C / D	10.46% / 17.27% / 16.65% / 7.12%	Best baseline: 11.09% / 26.12% / 24.44% / 6.61%	Shop D: both baselines beat Triody
Range coverage, day 14 / day 28 (target 80%)	A 87.8/87.8, B 91.5/95.1, C 70.7/79.3, D 84.2/97.6	n/a	Shop C runs narrow at day 14

Each forecast test runs from 82 starting days per shop. The starting days overlap, so the results are not independent. Settings were tuned on shop A only. Shop B was built before tuning. Shops C and D each exposed a problem the team then fixed, so neither counts as a fully clean test.

Evidence in `evidence/`: demo transcripts in English and Chinese, two audit logs with a chain check, the forecast backtest, the matcher evaluation and sandbox application files.

6. Known limits

- All data is synthetic. No real business has used Triody yet.
- The language-model planner was tested with scripted fake models, not a live model. `python -m triody llm-check` runs the demo plus six attack prompts once you add a model.
- In this prototype the owner device is a panel on the same dashboard. A real deployment needs SMS or a bank app push message.
- Without anchoring, a person with file access is able to rewrite the whole audit log with a valid chain, or delete entries from the end.
- Catalogue rates go out of date. Some products use a "from" rate or a market average, and replies say so.
- Triody gives information, not regulated financial advice. It makes no credit decisions.
- The full list of 26 risks, with likelihood, impact and mitigation, is in `docs/SECURITY_ASSESSMENT.md`.